

1 Transformer Model: Standard Architecture and Usage in LLaVA

1.1 The Standard Transformer

The Transformer is a neural network architecture for sequence transduction introduced by Vaswani et al. in *Attention Is All You Need* [4]. Unlike prior sequence models based on recurrent or convolutional networks, it relies exclusively on attention mechanisms to model dependencies between tokens, enabling full parallelization across positions during training [4]. In its original form, the architecture consists of an encoder stack and a decoder stack, as illustrated in Figure 1 [4].

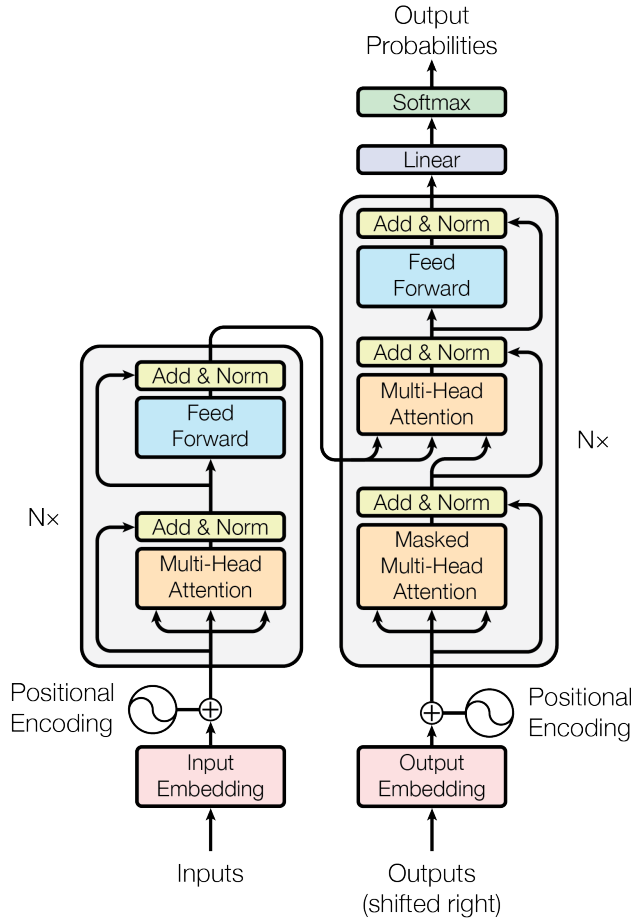


Figure 1: The Transformer encoder-decoder architecture with N stacked layers in each stack. [4].

1.1.1 Token Embeddings and Positional Encoding

The model processes each input as a sequence of token embeddings — fixed-size vectors representing individual tokens [4]. Since self-attention is permutation-invariant and produces identical outputs regardless of input order, positional information must be explicitly added before the first layer [4]. The original Transformer achieves this through sinusoidal positional encodings, defined for position pos and dimension index i as [4]:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (1)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

where d_{model} is the embedding dimension [4]. The sinusoidal form allows the model to generalize to sequence lengths not seen during training [4]. The final input to the first layer is:

$$\tilde{x}_{pos} = \text{Embedding}(\text{token}_{pos}) + PE_{pos} \quad (3)$$

1.1.2 Scaled Dot-Product Attention

The core operation is *Scaled Dot-Product Attention* [4]. Each token is linearly projected into three vectors — Query (Q), Key (K), and Value (V) — using learned weight matrices [4]. Attention weights are computed as scaled dot products between queries and keys, normalized by $\sqrt{d_k}$ to stabilize gradients, and passed through softmax [4]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (4)$$

where d_k is the key dimensionality [4]. The result is a weighted combination of value vectors, where weights reflect the relevance of each position to the current query [4].

1.1.3 Multi-Head Attention

Instead of computing a single attention map, the Transformer applies h parallel attention operations, each with distinct learned projections [4]. This allows each head to specialize in different types of token relationships simultaneously [4]:

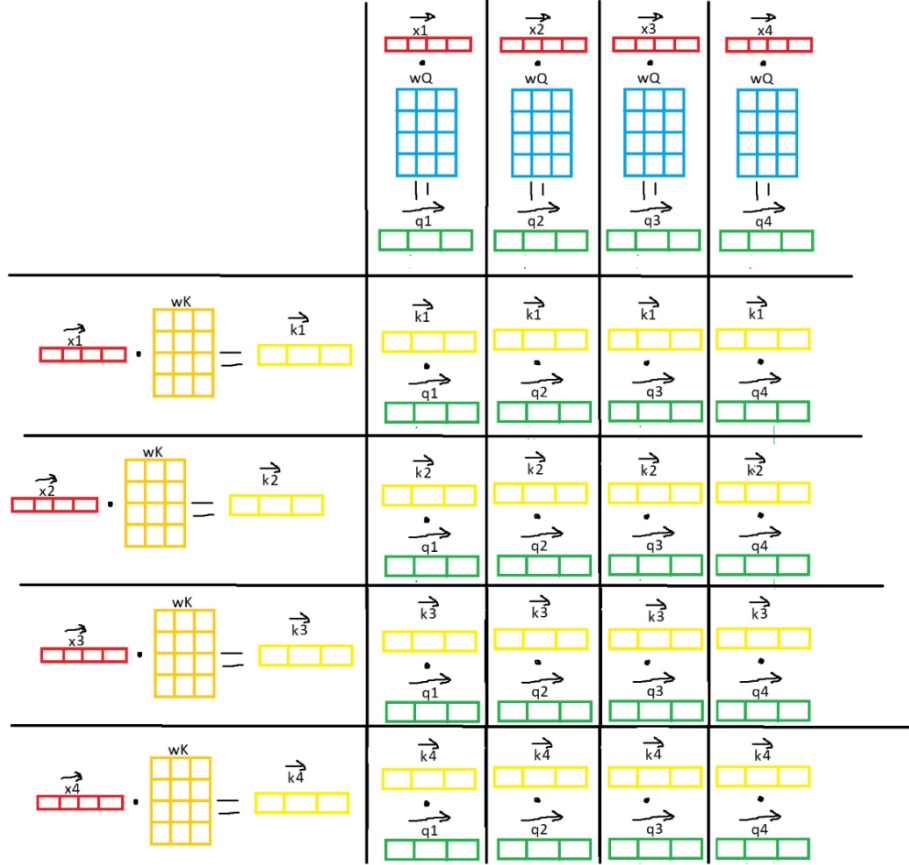


Figure 2: Query and Key vector projections across token positions. Each input token is multiplied by weight matrices W_K and W_Q to produce Key and Query vectors respectively. Every Query–Key pair is combined via dot product to compute the pairwise attention score, as defined in Equation 4.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (5)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (6)$$

where W_i^Q, W_i^K, W_i^V are learned per-head projections and W^O is an output projection [4]. In the original model, $h = 8$ with $d_k = d_v = d_{\text{model}}/h = 64$ [4].

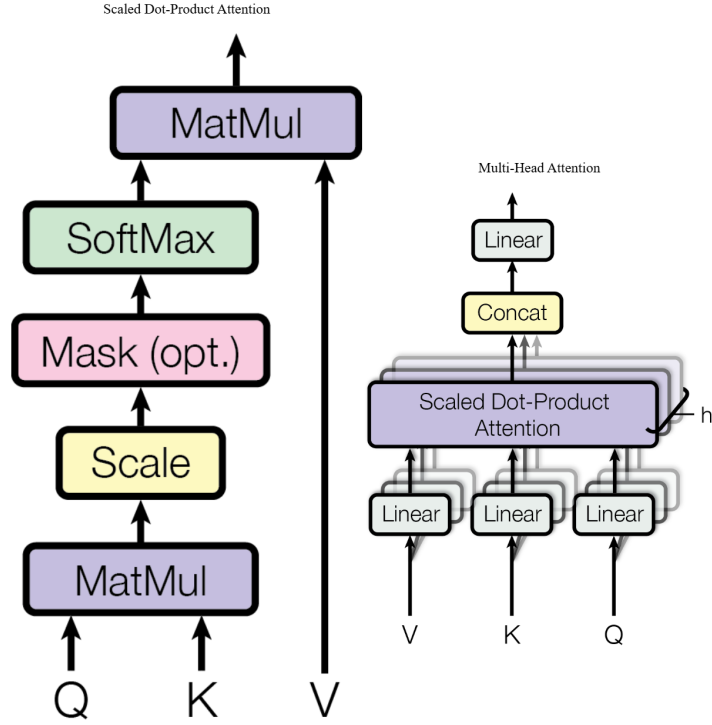


Figure 3: Scaled Dot-Product Attention (left) and Multi-Head Attention (right). [4].

1.1.4 Transformer Block Structure

Each Transformer block contains two sublayers: multi-head self-attention, and a position-wise feed-forward network (FFN) [4]. Both are wrapped with a residual connection followed by layer normalization [4]:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (7)$$

Layer normalization is defined as [4]:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta \quad (8)$$

where μ and σ^2 are the mean and variance over the embedding dimension, ϵ is a small stability constant, and γ, β are learned parameters [4]. This repeating block structure is used in both the encoder and decoder stacks [4].

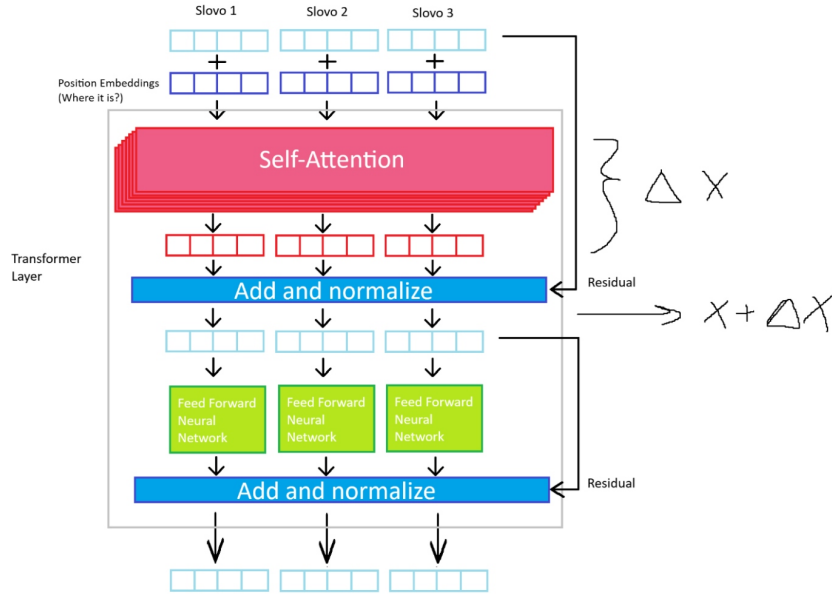


Figure 4: Multi-head Transformer block. Token embeddings combined with positional encodings pass through stacked self-attention heads and per-token Feed Forward Networks, each wrapped by residual connections and layer normalization (Equations 7–8).

1.1.5 Encoder and Decoder Stacks

The encoder is a stack of N identical blocks, each applying multi-head self-attention over the full input sequence [4]. The decoder stack also uses N identical blocks, but each block contains an additional **masked** self-attention sublayer (preventing attention to future positions) and an **encoder–decoder cross-attention** sublayer (where decoder queries attend over encoder keys and values) [4]. In the original model $N = 6$ for both stacks [4]. The full arrangement of both stacks is illustrated in Figure 1.

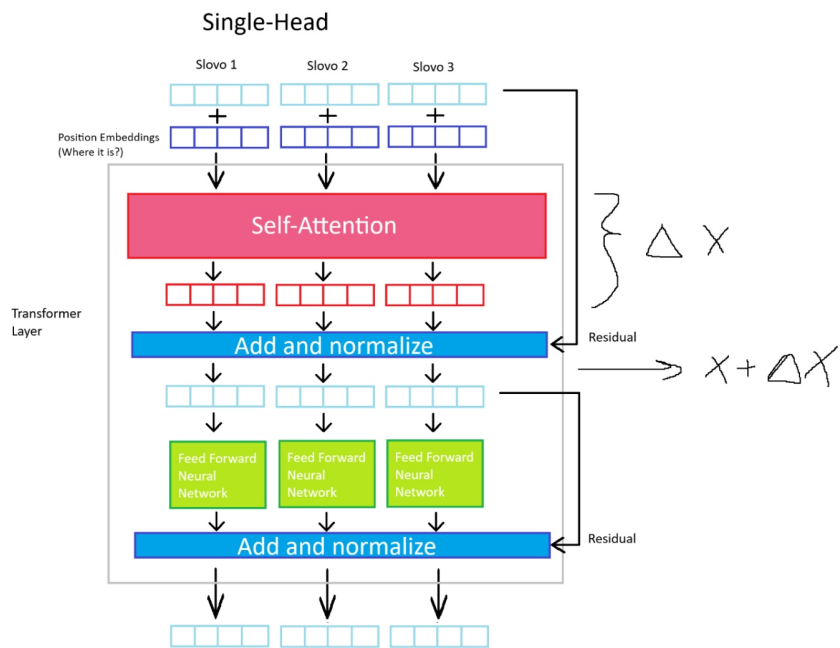


Figure 5: Single-head Transformer block. Structurally identical to the multi-head variant (Figure 4), but with a single self-attention operation instead of parallel heads [4].

1.1.6 Position-wise Feed-Forward Network

After the attention sublayer, each token passes independently through the same two-layer MLP [4]:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (9)$$

In the original model $d_{\text{model}} = 512$ with inner dimension $d_{ff} = 2048$ [4].

1.2 Differences Between the Standard Transformer and LLaVA

LLaVA (Large Language-and-Vision Assistant) builds on the Transformer paradigm but departs from the original encoder–decoder design in four key ways [3].

1.2.1 Two Separate Transformer Components

Where the original Transformer uses a single model for both encoding and decoding [4], LLaVA connects two independently pre-trained Transformer-based components: a Vision Transformer (ViT) encoder for visual input and a large language model (LLM) decoder for text generation [3]. Concretely, LLaVA-1.5 uses CLIP ViT-L/14 at 336×336 pixel resolution as the visual encoder and Vicuna-7B or 13B as the LLM [2].

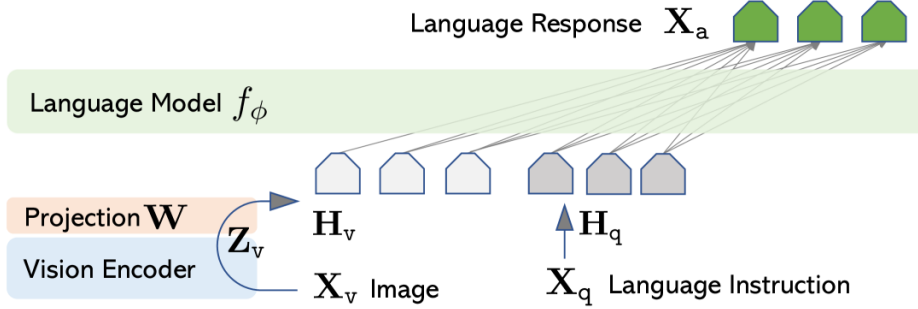


Figure 6: LLaVA network architecture. Visual features $Z_v = g(X_v)$ from the frozen ViT are projected via a trainable matrix W into the LLM embedding space as H_v , then processed jointly with text tokens. [3] (CC BY 4.0).

1.2.2 No Cross-Attention in the LLM

The original decoder conditions on encoder output through a dedicated cross-attention sublayer [4]. LLaVA omits this entirely: visual tokens are instead

concatenated with text tokens and passed as a single joint sequence into the LLM, which attends to both modalities through its standard masked self-attention [3][1].

1.2.3 MLP Projector for Modality Alignment

Because the ViT and the LLM operate in different embedding spaces, LLaVA introduces a trainable projector to map visual features into the LLM’s embedding dimension [3]. The original LLaVA uses a linear projection [3]:

$$H_v = W \cdot Z_v, \quad Z_v = g(X_v) \quad (10)$$

LLaVA-1.5 replaces this with a two-layer MLP with GELU activations for improved alignment [2]:

$$H_v = W_2 \text{GELU}(W_1 Z_v + b_1) + b_2 \quad (11)$$

This module has no equivalent in the original Transformer, which assumes a unified token interface between encoder and decoder [4].

1.2.4 Two-Stage Training with Component Freezing

The original Transformer is trained end-to-end from scratch [4]. LLaVA instead uses a two-stage process [3]:

- **Stage 1 — Feature alignment:** Only the projector is trained; the ViT encoder and LLM remain frozen, teaching the projector to produce visual tokens compatible with the frozen LLM [3].
- **Stage 2 — Instruction fine-tuning:** The projector and LLM are jointly trained on visual instruction data, while the ViT encoder remains frozen throughout [3].

1.3 Structural Comparison

1.4 Connection to Layer-wise Activation Analysis

Because LLaVA’s LLM processes visual and text tokens jointly through the same self-attention mechanism, the hidden states at each layer encode a mixture of visual and linguistic information [1]. Representations become increasingly abstract with depth, transitioning from low-level patch features to high-level semantic concepts [2]. This layer-by-layer structure makes it possible to extract intermediate hidden states across the full depth of the

Table 1: Original Transformer vs. LLaVA-1.5.

Aspect	Original Transformer [4]	LLaVA-1.5 [3, 2]
Core structure	Single encoder-decoder	Separate ViT encoder + LLM decoder
Conditioning mechanism	Decoder cross-attention over encoder outputs	Masked self-attention over joint visual + text tokens
Token types	Text only	Text + image-patch visual tokens
Alignment module	Not required	2-layer MLP projector with GELU
Training procedure	End-to-end	Stage 1: projector; Stage 2: projector + LLM
Frozen components	None	ViT frozen throughout; LLM frozen in Stage 1

network, which forms the basis of the activation extraction and clustering experiments in this thesis [3].

References

- [1] Hugging Face. Llava - transformers documentation, 2024. Accessed: 2026-03-09.
- [2] LearnOpenCV. Llava architecture: From frozen vit to fine-tuned llm, 2023. Accessed: 2026-03-09.
- [3] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.